

Manejo e Implementación de Archivos



Semana #4



Guatemala, 29 de julio de 2026

Ing. Luis Aguilar

Objetivos

Al finalizar la clase el estudiante será capaz de:

1. Comprender el trabajo colaborativo.
2. Utilizar repositorios remotos.
3. Clonar un proyecto.
4. Enviar cambios al servidor.
5. Descargar cambios. Trabajar con ramas. Crear un Pull Request.

¿Qué es el trabajo colaborativo?

Es la forma en que varias personas desarrollan un mismo proyecto sin sobrescribir el trabajo de otros.



Beneficios



Organización

Estructura clara de archivos y tareas.



Historial

Registro de cambios y versiones.



Seguridad

Respaldo y control de la información.



Trabajo simultáneo

Varios miembros pueden trabajar al mismo tiempo.

Problemas sin Git



Sin control de versiones



Cambios sobrescritos



Archivos duplicados y confusión



Pérdida de tiempo y errores

Git vs GitHub vs GitLab

Herramientas que trabajan juntas para el desarrollo colaborativo

GIT



Sistema de control de versiones distribuido que registra cambios en el código fuente.

¿PARA QUÉ SIRVE?

-  Guardar historial de cambios
-  Trabajar en ramas (branches)
-  Volver a versiones anteriores

GITHUB



Plataforma en la nube para alojar repositorios Git y colaborar con otros desarrolladores.

¿PARA QUÉ SIRVE?

-  Alojar repositorios remotos
-  Revisar código y colaborar
-  Integraciones y automatizaciones

GITLAB



Plataforma de DevOps completa para alojar repositorios Git y gestionar todo el ciclo de desarrollo.

¿PARA QUÉ SIRVE?

-  Repositorio Git y gestión de código
-  CI/CD integrado
-  Seguridad y gestión de proyectos



GIT

Gestiona el código en tu computadora



GITHUB

Almacena el código en la nube y permite colaborar



GITLAB

Ofrece colaboración + herramientas DevOps en un solo lugar

ARQUITECTURA



Computadora Local



Repositorio Local



Repositorio Remoto
(GitLab)

REPOSITORIO REMOTO



Es una copia central del proyecto.

Permite:



Compartir



Respaldar



Colaborar

FLUJO BÁSICO



Editar
archivos



git add



git commit



git push



Repositorio
remoto

DESCARGAR CAMBIOS



git pull

Obtiene los cambios
realizados por otros
integrantes.

CLONAR UN PROYECTO



git clone URL

Ejemplo:

```
git clone https://gitlab.com/  
usuario/proyecto.git
```

RESUMEN DEL FLUJO COMPLETO



Trabajas en tu
computadora



Repositorio
local



Repositorio remoto
(GitLab)



Trabajas, commit, envías (push) y compartes.
Descargas (pull) para mantenerte actualizado.

¿QUÉ OCURRE AL CLONAR?

Se descarga:



Historial

Todos los commits y cambios realizados en el proyecto.



Ramas

Todas las ramas existentes en el repositorio.



Archivos

Todos los archivos del proyecto.



Configuración

Archivos y ajustes del repositorio.

BRANCH (RAMA)

Una rama permite desarrollar nuevas funciones sin afectar la versión principal.



Trabajo aislado

Seguro

Colaborativo

¿POR QUÉ USAR RAMAS?



Evitar errores

Las nuevas funciones se desarrollan sin arriesgar la versión principal.



Trabajar varias personas

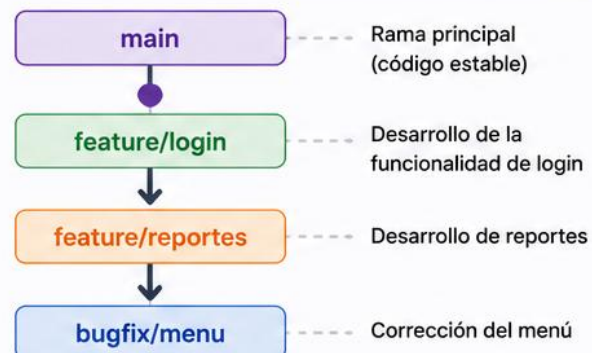
Cada integrante trabaja en su propia rama sin interferir con los demás.



Desarrollar funcionalidades independientes

Permite avanzar en varias tareas al mismo tiempo.

RAMA PRINCIPAL Y RAMAS DE TRABAJO



CREAR UNA RAMA

```
git checkout -b feature-login
```

Crea y cambia a la nueva rama.



CAMBIAR DE RAMA

```
git checkout main  
git checkout feature-login
```

Cambia a otra rama existente.



VER RAMAS

```
git branch
```

Muestra todas las ramas locales.



ENVIAR UNA RAMA

```
git push origin feature-login
```

Envía la rama al repositorio remoto.



FLUJO TÍPICO CON RAMAS



Crear rama



Hacer cambios



Commit



Push



Pull Request



Merge



Integrado a main



PULL REQUEST



Solicitud para integrar una **rama** hacia **otra**.

¿QUÉ CONTIENE UN PULL REQUEST?



Descripción

Explica el propósito y los cambios realizados.



Archivos modificados

Lista de archivos cambiados, agregados o eliminados.



Comentarios

Conversación entre los miembros sobre los cambios.



Revisión

Los revisores analizan el código y dan retroalimentación.



Aprobación

Los revisores aprueban los cambios para integrarlos.



Objetivo: asegurar que los cambios sean correctos, compatibles y de calidad antes de integrar a la rama principal.

Merge Request !27 Open

Overview Commits 3 Changes 5

+15 -2

Descripción

Archivos modificados

Comentarios

Revisión

Aprobación

FLUJO COMPLETO



1. Crear rama

Crea una rama desde la rama principal para desarrollar la nueva funcionalidad.



2. Modificar archivos

Realiza los cambios necesarios en los archivos del proyecto.



3. Commit

Guarda los cambios en el repositorio local con un mensaje descriptivo.



4. Push

Envía tu rama y los cambios al repositorio remoto (GitLab).



5. Pull Request

Crea un Pull Request para solicitar la integración de tu rama a otra (usualmente main).



6. Revisión y aprobación

El equipo revisa los cambios, comenta y aprueba si todo está correcto.



7. Merge

Se integran los cambios a la rama destino. La nueva funcionalidad ya forma parte del proyecto.



Resultado: cambios integrados de forma segura, revisada y colaborativa.



TIPOS DE MERGE MÁS COMUNES EN PULL REQUEST

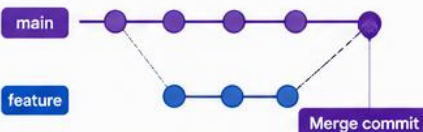


Al crear un Merge Request en GitLab, puedes elegir cómo integrar los cambios de tu rama a la rama destino.



1. MERGE COMMIT (Predeterminado)

Crea un commit de merge que combina el historial de ambas ramas.



✓ Ventajas

- Conserva todo el historial.
- Fácil de entender el contexto del merge.
- Útil para auditoría y rastreo.

✗ Desventajas

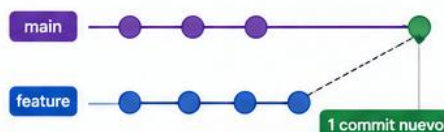
- Historial más "ruidoso" con commits de merge.
- Puede verse menos lineal.

`>_` Comando local equivalente:
`git merge feature/mia_semana4`



2. SQUASH AND MERGE

Combina todos los commits de la rama en uno solo antes de hacer el merge.



✓ Ventajas

- Historial limpio y fácil de leer.
- Agrupa todos los cambios en un solo commit.
- Ideal para features pequeñas o medianas.

✗ Desventajas

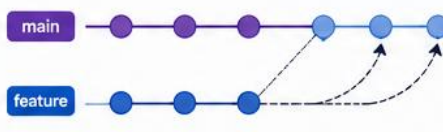
- Se pierde el detalle de los commits intermedios.
- Menos información para depuración.

`>_` Comando local equivalente:
`git merge --squash feature/mia_semana4`
`git commit`



3. REBASE AND MERGE

Reproduce (reaplica) los commits de la rama sobre la rama destino y luego hace el merge.



✓ Ventajas

- Historial lineal y limpio.
- No crea commits de merge adicionales.
- Útil en proyectos donde se valora la linealidad.

✗ Desventajas

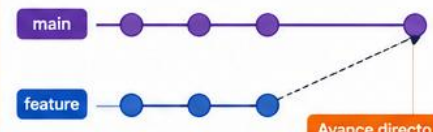
- Reescribe el historial de la rama.
- Puede causar conflictos si la rama fue compartida.

`>_` Comando local equivalente:
`git rebase main`
`git checkout main && git merge feature/mia_semana4`



4. FAST-FORWARD MERGE

Solo avanza la rama destino cuando no hay divergencia.



✓ Ventajas

- Historial lineal y muy limpio.
- Sin commits adicionales.
- El más simple y rápido.

✗ Desventajas

- Solo funciona si no hay divergencia entre ramas.

`>_` Comando local equivalente:
`git merge --ff-only feature/mia_semana4`



¿CUÁNDO USAR CADA TIPO?

Merge Commit

- Proyectos grandes, necesidad de trazabilidad completa.

Squash and Merge

- Features pequeñas/medianas, historial limpio y claro.

Rebase and Merge

- Equipos con flujo lineal (ej. GitFlow simplificado, trunk based).

Fast-forward Merge

- Cambios pequeños o ramas sin divergencias.

COMPARACIÓN VISUAL

Tipo	Antes del Merge	Después del Merge
1. Merge Commit		
2. Squash and Merge		
3. Rebase and Merge		
4. Fast-forward Merge		



NOTAS IMPORTANTES

- Antes de crear un Merge Request, asegúrate de que tu rama esté actualizada.

```
git pull origin main
```

- Resuelve los conflictos localmente antes de enviar.
- Sigue las convenciones de commits:

```
feat: agrega información personal al README
```

- El tipo de merge se selecciona en GitLab antes de confirmar el Merge Request.



Recomendación general:

Usa Squash and Merge para mantener un historial limpio, a menos que necesites trazabilidad completa.



Convenciones de ramas:

`feature/mia_semana4` → nuevas funcionalidades
`bugfix/descripcion` → corrección de errores

`hotfix/descripcion`
`docs/descripcion`

→ correcciones urgentes
→ documentación

✓ BUENAS PRÁCTICAS



Commits pequeños

Realiza cambios en porciones pequeñas y con un propósito claro.



Mensajes claros

Describe el propósito de cada commit de forma concisa.



Una tarea por rama

Trabaja en una funcionalidad por rama. Evita mezclar tareas.



Actualizar antes de trabajar

Trae los últimos cambios de la rama base antes de comenzar.



Revisa antes de fusionar

Usa Pull Requests y revisiones de código antes de hacer merge.

📁 CONVENCIONES

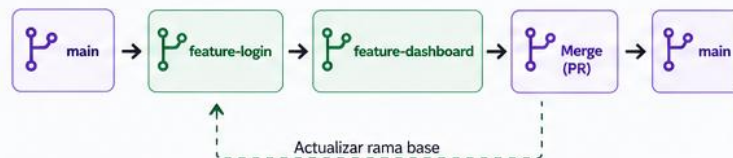
ESTÁNDAR DE BRANCH NAMES

feature/	Nuevas funcionalidades.
bugfix/	Corrección de errores.
hotfix/	Arreglos urgentes en producción.
docs/	Documentación.
refactor/	Mejoras de código sin cambiar funcionalidad.
test/	Pruebas automatizadas o experimentos.
chore/	Tareas de mantenimiento, dependencias, configuración.

OTRAS CONVENCIONES

abc	Nombres en minúsculas Usa guiones medios (-) para separar palabras.
	Ramas desde la base correcta Crea ramas desde main (o develop) actualizada.
	Pull Requests (PR) Abre un PR para integrar cambios.
	Protege ramas principales Usa reglas de protección en main y release/*.
	Elimina ramas ya fusionadas Mantén el repositorio limpio y organizado.
	Versionado Usa etiquetas (tags) para releases.

EJEMPLO DE FLUJO



ESTÁNDAR DE COMMIT MSG

Usa el formato Conventional Commits.

```
<tipo>(<ámbito opcional>): <descripción corta>
[ cuerpo opcional ]
[ pie opcional ]
```

Tipos permitidos:

feat	Nueva funcionalidad	refactor	Refactorización de código
fix	Corrección de errores	test	Pruebas
docs	Documentación	chore	Tareas de mantenimiento
style	Estilos (formato, espacios, etc.)	perf	Mejoras de rendimiento

Ejemplos:

- feat(auth): agregar inicio de sesión con Google
- fix(user): corregir validación de correo
- docs(readme): actualizar guía de instalación
- refactor(api): simplificar lógica de autenticación

CONVENCIONES ADICIONALES

	Rebase vs Merge Usa rebase para mantener historial limpio y lineal.
</>	Código consistente Sigue linters, formateadores y estándares del proyecto.
	Pruebas Escribe y ejecuta pruebas antes de hacer PR.
	Integración Continua (CI) Asegúrate de que el pipeline pase antes de merge.
	Documenta cambios importantes Actualiza README, CHANGELOG y docs.
	Comunicación Comenta en los PRs, responde y colabora.

✓ MÁS BUENAS PRÁCTICAS PARA REPOSITORIOS REMOTOS



Protege información sensible

Usa variables de entorno y secretos (no subir claves al repositorio).



Usa releases y etiquetas

Marca versiones estables con tags semánticos.



Ambientes y ramas

Usa ramas de ambiente (ej. develop, staging, production).



Historial limpio

Evita commits grandes, merge innecesarios y WIP en main.



Revisión de código

Al menos un revisor antes de integrar cambios.



Mantén el repo limpio

Elimina ramas y archivos innecesarios. Usa .gitignore.



GIT WORKFLOW

Modelos de flujo de trabajo Git más utilizados



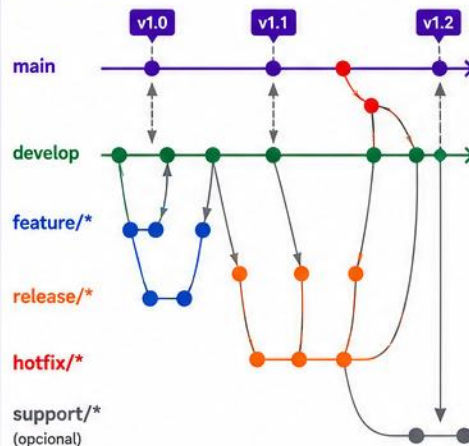
TIPOS DE RAMAS

- main / trunk** (producción)
- develop** (integración)
- feature/*** (nuevas funcionalidades)
- release/*** (preparación de release)
- hotfix/*** (correcciones urgentes)
- support/*** (soporte, opcional)

CONVENCIONES DEL DIAGRAMA

- Rama (línea de vida)
- Branch (crear rama)
- Merge (integración) (normalmente vía PR)
- Tag / Release (versión publicada)
- Commit (punto importante)

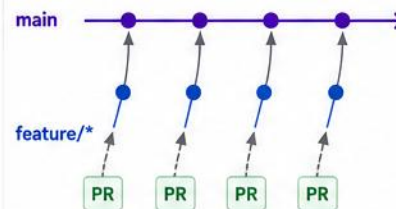
1 GITFLOW WORKFLOW



¿CUÁNDO USAR?

Proyectos grandes con versiones planificadas y múltiples releases. Recomendado cuando se requiere estabilidad y soporte a versiones.

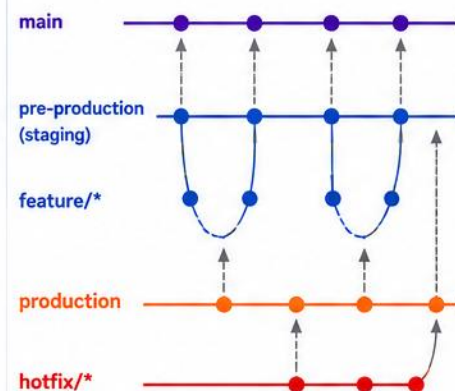
2 GITHUB FLOW



¿CUÁNDO USAR?

Equipos ágiles que hacen despliegue continuo. Ideal para aplicaciones web y SaaS.

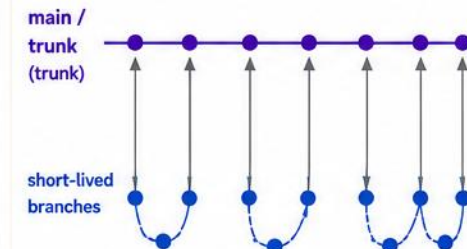
3 GITLAB FLOW



¿CUÁNDO USAR?

Equipos que despliegan por ambientes (Pre-producción / Producción). Permite control intermedio antes del release final.

4 TRUNK-BASED DEVELOPMENT



¿CUÁNDO USAR?

Equipos que buscan máxima velocidad y automatización. CI/CD maduro. Múltiples despliegues al día.



RECUERDA: Las flechas muestran el flujo del código entre ramas. No representan comandos (push, pull, fetch), sino la creación e integración de ramas.



Branch: creación de una rama



Merge: integración de cambios (vía PR)



Tag: publicación de una versión

1 CLONAR REPOSITORIO

Clona el repositorio asignado desde GitLab.



```
$ git clone
https://gitlab.com/
grupo/curso-manejo-
archivos.git
```



Esto creará una carpeta local con todo el historial del proyecto.

2 CREAR RAMA

Crea una rama para tu práctica.



```
$ git checkout -b
feature/mia_semana4
```

Convención de nombres:

feature/<nombre>_<semana#>

Ejemplo: feature/mia_semana4

3 EDITAR ARCHIVO

Modifica un archivo del proyecto.



README.md

Agregar en el archivo:

- Nombre
- Carné
- Fecha
- Comentario



Guardar cambios

4 GUARDAR CAMBIOS

Agrega y confirma tus cambios.

```
$ git add .
$ git commit -m
"feat: agrega información
personal en README.md"
```

Convención de mensajes (Commits):

<tipo>: <descripción breve y clara>

Tipos comunes:

feat nueva funcionalidad
fix corrección de errores
docs cambios en documentación
chore tareas de mantenimiento
refactor mejora de código

Ejemplos:

- ✓ feat: agrega login de usuario
- ✓ fix: corrige validación de fecha
- ✓ docs: actualiza README
- ✓ chore: actualiza dependencias

5 PUBLICAR CAMBIOS

Envía tu rama al repositorio remoto.



```
$ git push origin
feature/mia_semana4
```



Ahora tu rama está disponible en GitLab para el resto del equipo.

6 CREAR PULL REQUEST

Desde GitLab, solicita integrar tu rama.



- 1 Ve a tu proyecto en GitLab
- 2 Clic en "Merge requests"
- 3 Clic en "New merge request"
- 4 Selecciona la rama origen y destino

From (origen)

feature/mia_semana4

To (destino)

main

7 DETALLE DEL PULL REQUEST

Completa la información del Merge Request.

Title

feat: agrega información personal en README.md

Description

Descripción
Se agrega mi información personal en el archivo README.md
- Nombre
- Carné
- Fecha
- Comentario
Relacionado con la práctica de la semana 4.



Revisa que la descripción sea clara y muestre el propósito del cambio.

Adjuntar evidencia (opcional)

Create merge request

8 REVISION Y APROBACIÓN

Tu equipo revisará los cambios.



Revisión del código



Comentarios



Aprobación



Merge (integración)



Una vez aprobado, se integra a la rama main.

9 ERRORES COMUNES

- ✗ Olvidar hacer pull antes de trabajar.
- ✗ Trabajar directamente sobre main.
- ✗ No hacer commit antes del push.
- ✗ Resolver conflictos incorrectamente.
- ✗ Mensajes de commit poco claros.



Revisar y seguir el flujo evita conflictos y pérdida de trabajo.

10 ACTIVIDAD EN CLASE

Cada estudiante deberá:



- ✓ Clonar el repositorio.
- ✓ Crear una rama con su nombre (feature/mia_semana4).
- ✓ Agregar un archivo README.md con su información.
- ✓ Hacer commit con mensaje según convención.
- ✓ Publicar la rama.
- ✓ Crear un Merge Request con título y descripción claros.

11 RECURSOS ÚTILES



Documentación Git
git-scm.com/doc



Ayuda GitLab
docs.gitlab.com/ee/



Comandos Git
git --help

12 RECUERDA

- ✓ Commits pequeños y frecuentes.
- ✓ Mensajes claros y con propósito.
- ✓ Una rama por tarea.
- ✓ Comunicación con tu equipo.
- ✓ Revisar antes de integrar.



Buen trabajo colaborativo = mejor código y mejores resultados.

FLUJO VISUAL DEL PROCESO



Clonar repositorio



Crear rama



Editar archivo



Commit (guardar cambios)



Push (publicar rama)



Pull Request (solicitar integración)



Revisión y aprobación



Merge (integrar a main)



Listo ¡Buen trabajo!